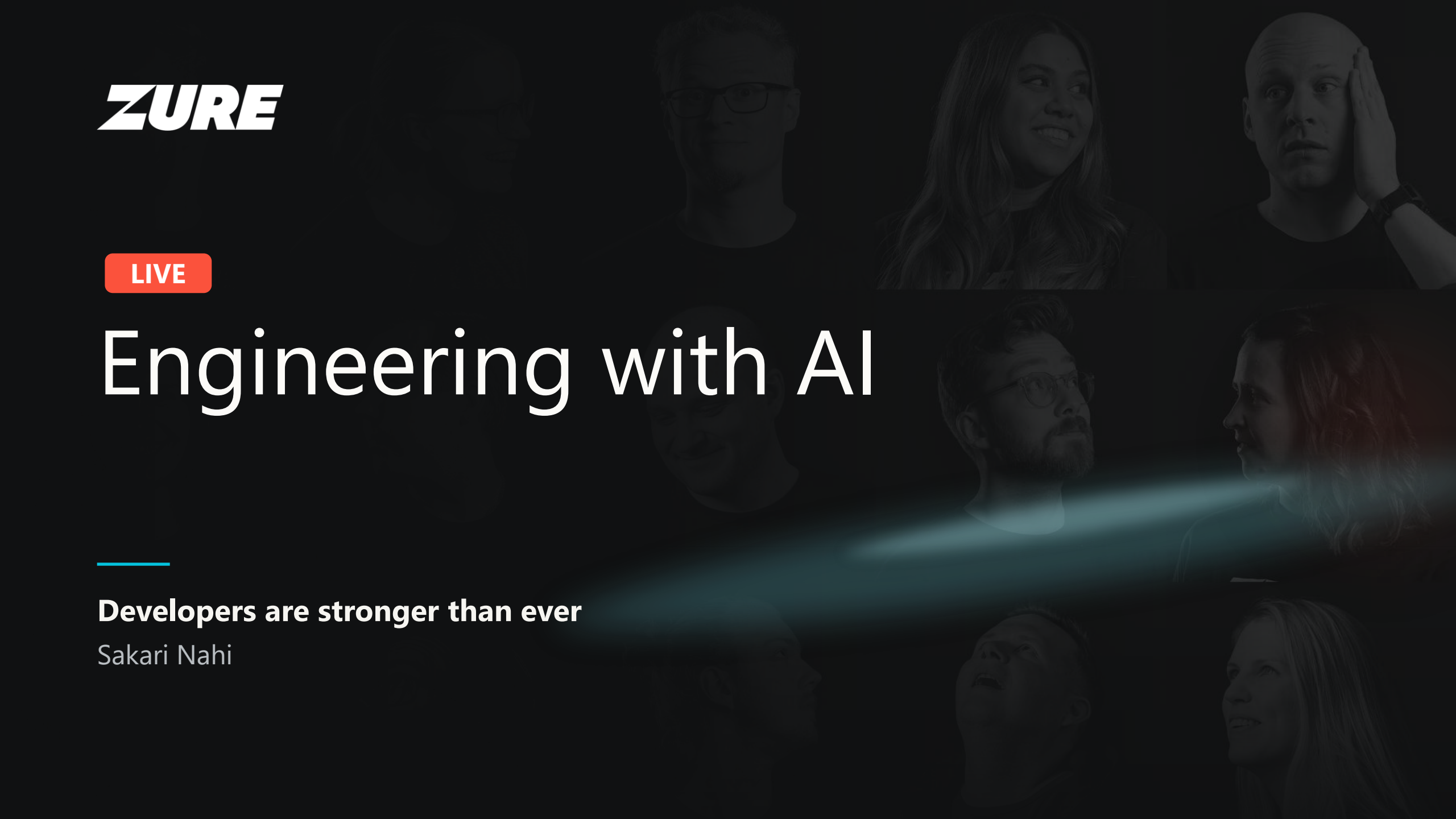
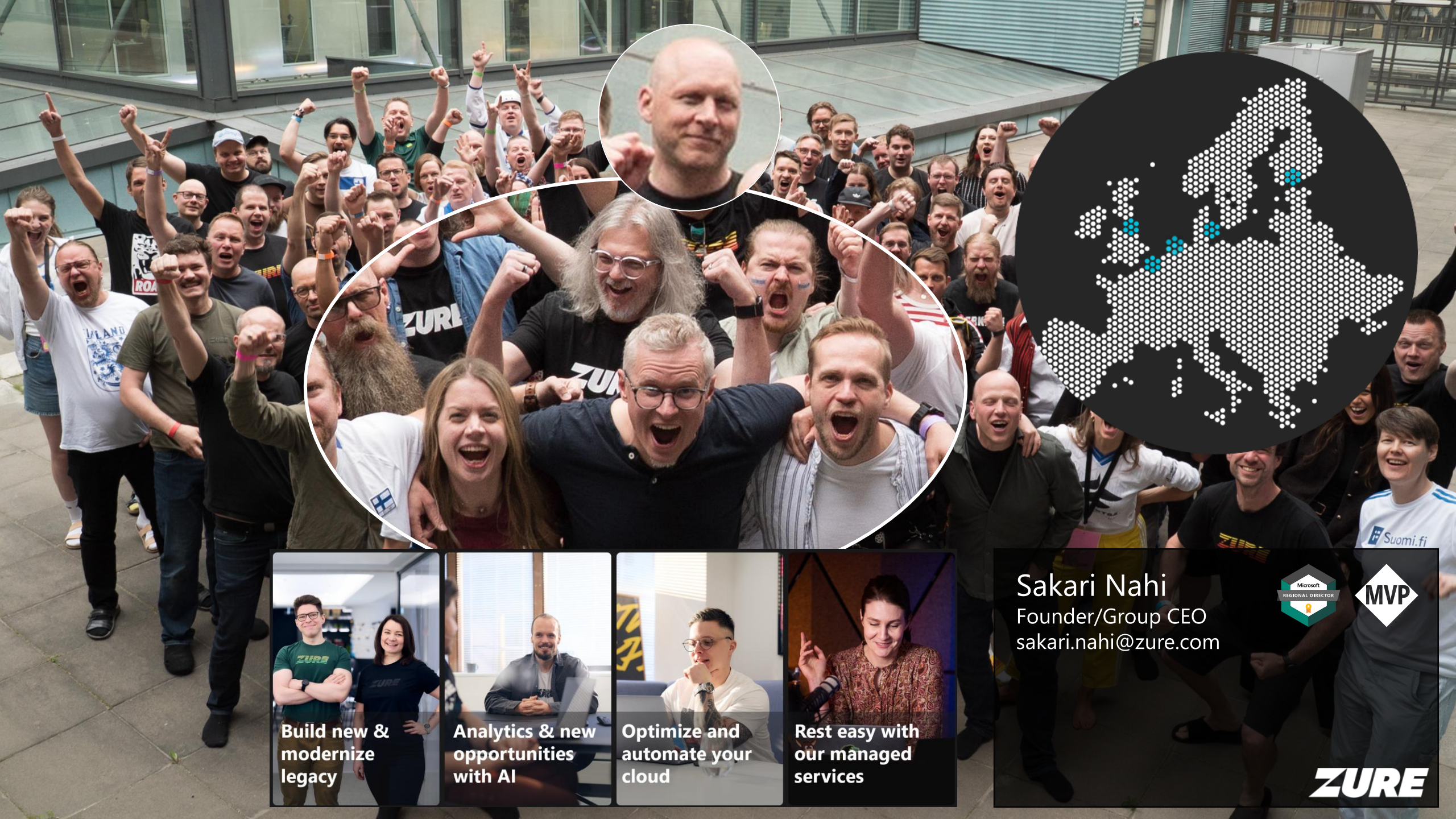

ZURE
LIVE
Engineering with AI
Developers are stronger than ever
Sakari Nahi

5 agent demonstration Start.sh

ZURE



Sakari Nahi
Founder/Group CEO
sakari.nahi@zure.com

Microsoft REGIONAL DIRECTOR

MVP

ZURE

Our job has changed

Spec-driven dev demo start

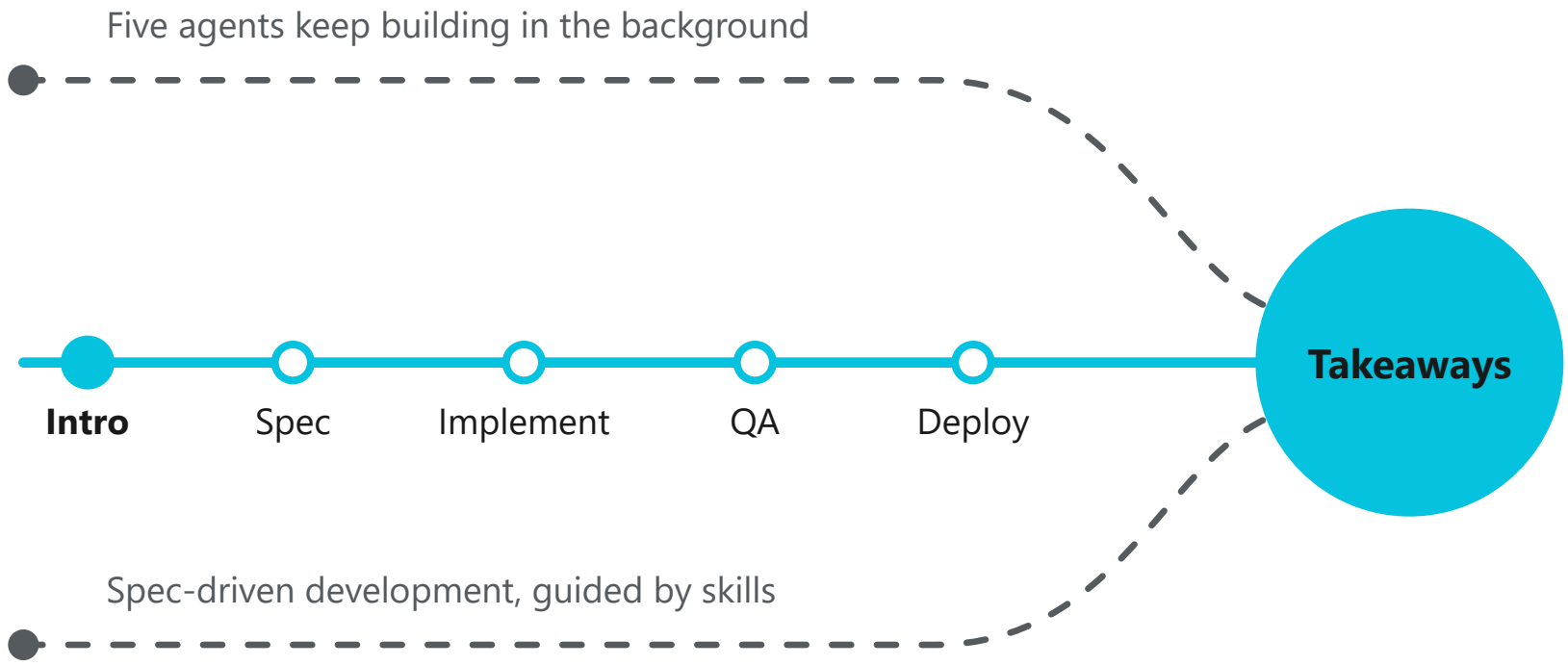
ZURE

Today

5-agent demo
Started first

Slides
6 parts, tracker bottom right

SDD demo
Started just before



Along the way: short side quests to the browser for the little demos

I wish I had a silver bullet

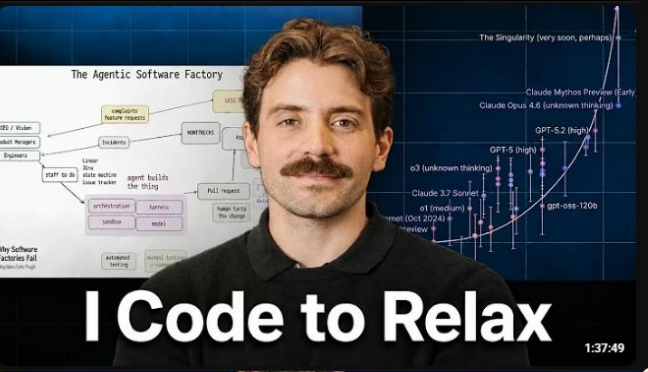
ZURE

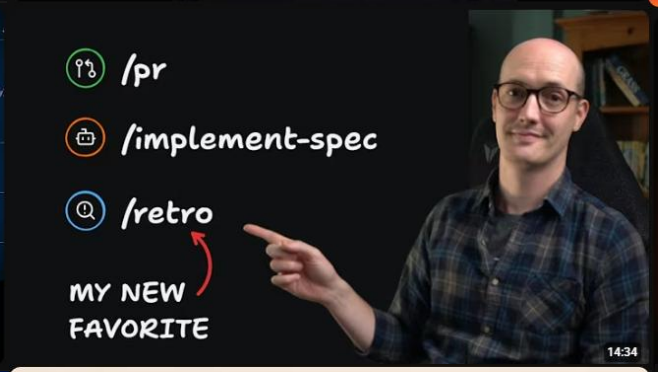

AI Engineer World's Fair
humanlayer
Read the Code!
19:18


AI Engineer World's Fair
UC Berkeley
ONTOLOGY
Ontologies Keep Agents Honest
21:18

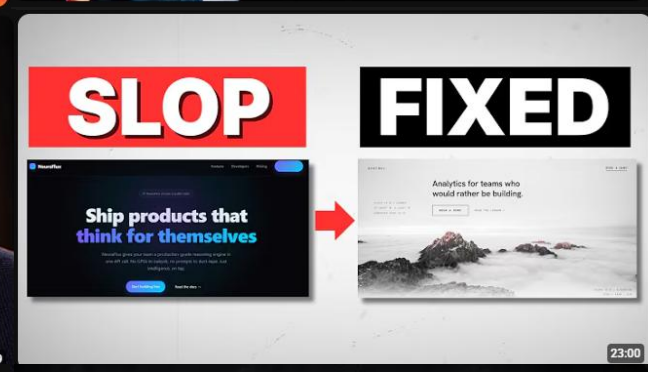

Stop building zombie UIs.
LENNY & FRIENDS SUMMIT
Katie Dill
Head of Design, Stripe
21:48


state of the tech industry in 2026
author, The Pragmatic Engineer
– GERGELY OROSZ
29:39


The Agentic Software Factory
I Code to Relax
1:37:49


/pr
/implement-spec
/retro
MY NEW FAVORITE
14:34


The Rules Have Changed
Skill.md
12:49


SLOP
FIXED
Ship products that think for themselves
Analytics for teams who would rather be building
23:00


You're NOT ALONE
18:40


FREE RESET
OPUS 5.5
12 New Rules
7:03


Uncle Bob
on Software Fundamentals and AI
56:40


agentic engineering
loops
worktree
validation
skills
memory
review
parallel
plan
tmux
workflow
html
whisper
orchestration
Watch Me Code
45:46


CLAUDE CODE
SENIOR DEV WORKFLOW
9:47


PROMPTING IS DEAD
1:43:03


Steal Their Process
7:01


Podcast
More Context Makes The Agent Dumber
HumanLayer's Dexter Horthy
AINativeDev
56:12

FAROS

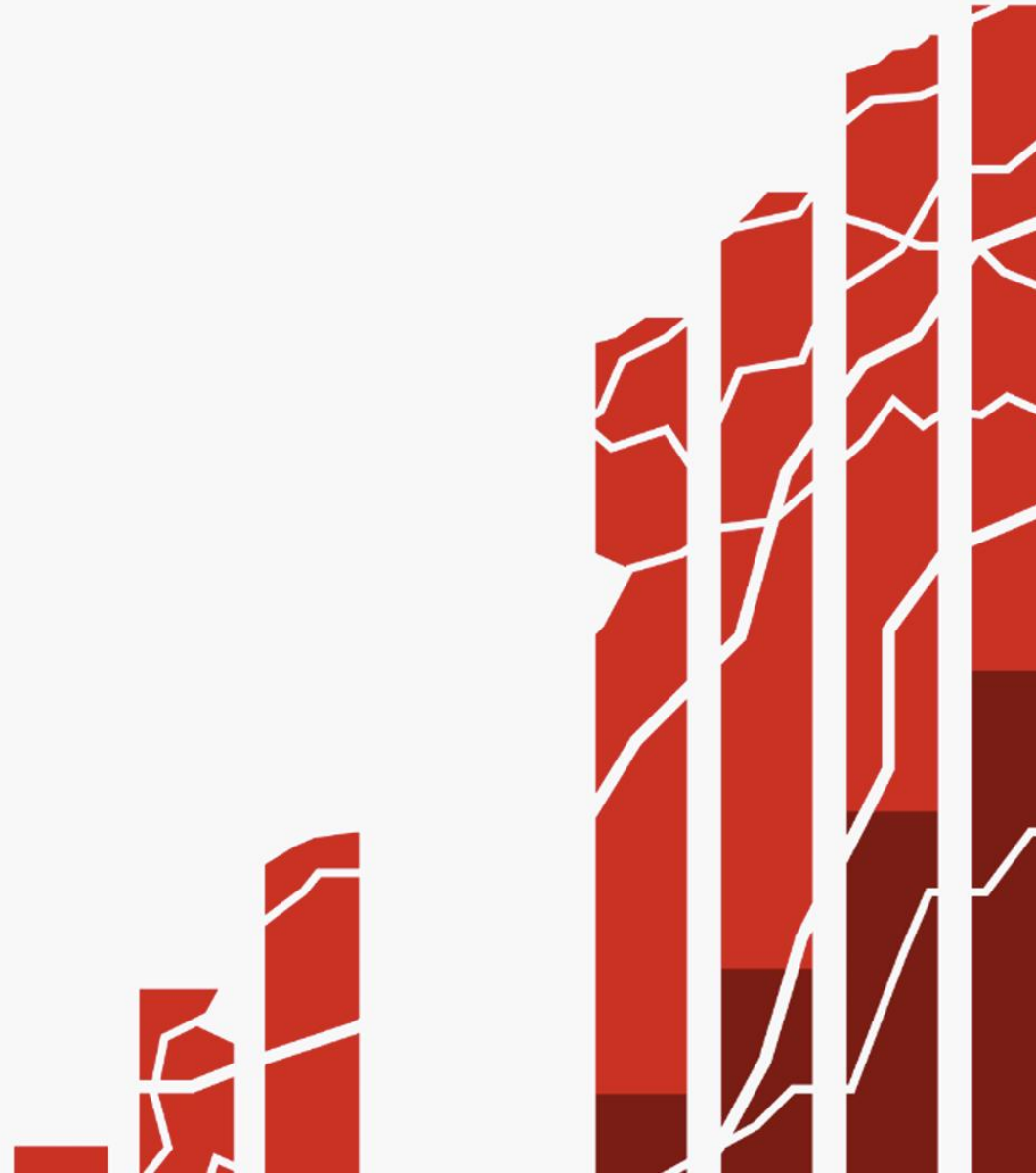
AI ENGINEERING REPORT Q3 2026

The Speed Trap

The industry kept accelerating. The controls didn't keep up.

Delivery is accelerating, but teams are getting there by skipping review, and the cost is landing downstream in QA and operations.

Analysis of 12 months of data from 22,000 developers on how engineering organizations are handling heightened AI usage at scale.



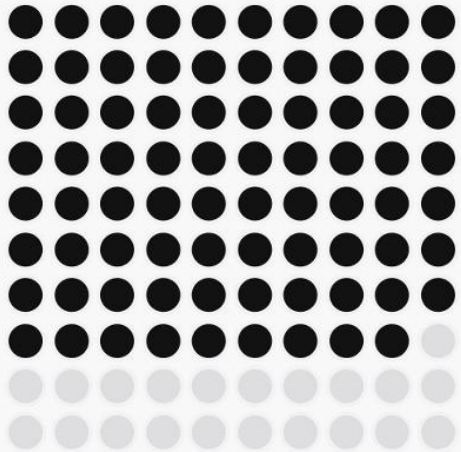
Faros: AI Engineering Report Q3 2026 - The Speed Trap

UTILIZATION IS WON. THE FRONTIER MOVED TO AGENTS.

State of AI adoption in software engineering

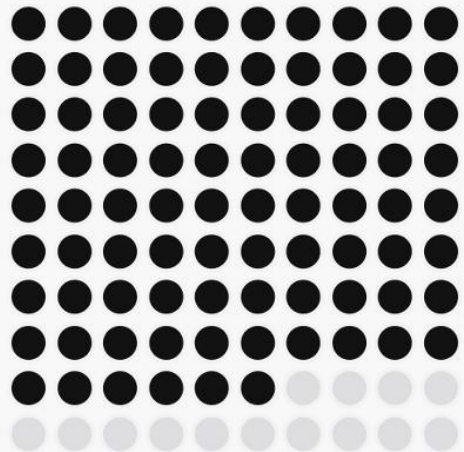
+79%
of developers

use at least one AI tool weekly
up from **+60%** in prior dataset



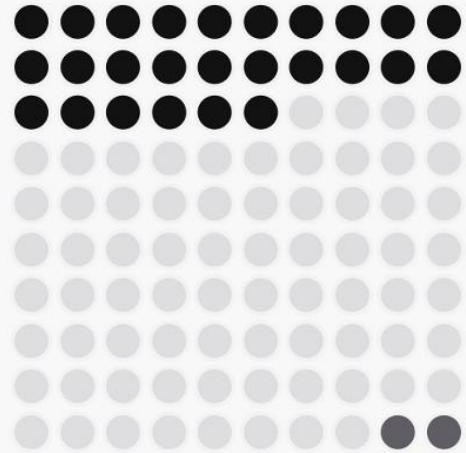
+86%
of teams

exceed the 50% weekly active user threshold
up from **+80%** in prior dataset



+26%
of PRs

are reviewed by an AI agents
up from **+25%** in prior dataset



2% of PRs are opened by AI agents
up from <1% in prior dataset

Telemetry dataset size: 22,000 developers across 4,000 teams

FAROS

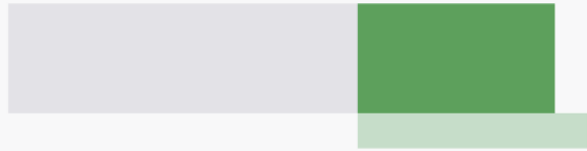
ZURE

OUTPUT IS UP. NOT ALL OF IT STICKS.

Impacts of high AI adoption on engineering throughput

Epic

A sizeable feature of capacity, hundreds of tasks

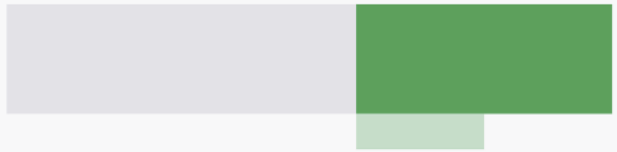


+53.7%

Epics completed per developer
down from +66.2% in prior dataset

Task

Individual work item, may span multiple PRs



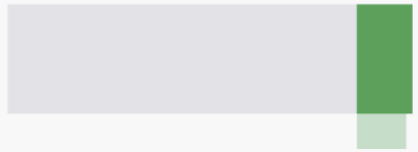
+69.0%

Task throughput per developer
up from +33.7% in prior dataset

Within this, tasks with an associated PR grew 127.3% per team. Code-specific work accelerated far faster than overall task volume.

Pull Request (PR)

Atomic unit of code change

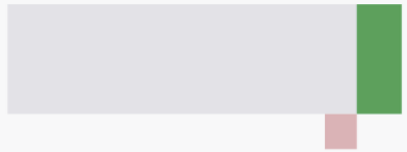


+16.6%

PR merge rate per developer
up from +16.2% in prior data set

Deployment

Ships code changes to production

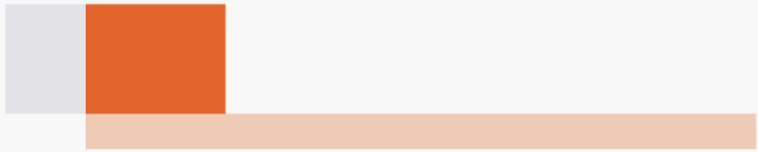


+13.8%

Deployments per week
up from -11.7% in prior dataset

Code Churn

Lines deleted to lines added per quarter, for merged code



+71.6%

Code deletion ratio
down from +861% in prior dataset

Telemetry dataset size: 22,000 developers across 4,000 teams

AI Engineering Report Q3 2026 | The Speed Trap

FARROS

ZURE

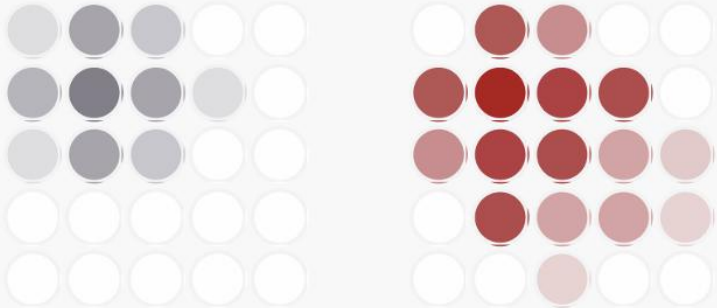
CODE CHANGES ARE LARGER AND REACH FURTHER

Impacts of high AI adoption on code size and spread

+52.0%

files edited per PR

down from +59.7% in prior dataset



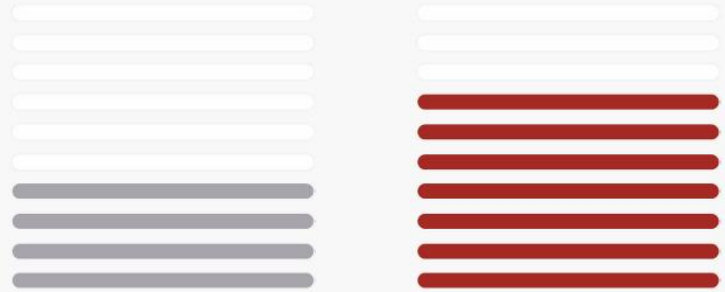
LOW AI ADOPTION

HIGH AI ADOPTION

+71.8%

average PR size

up from +51.3% in prior dataset



LOW AI ADOPTION

HIGH AI ADOPTION

DEVELOPER FOOTRPINT/MONTH

+80.8%

filed touched per developer

down from +149.9.% in prior dataset

+7.0%

repositories touched per developer

down from +11.7% in prior dataset

Telemetry dataset size: 22,000 developers across 4,000 teams

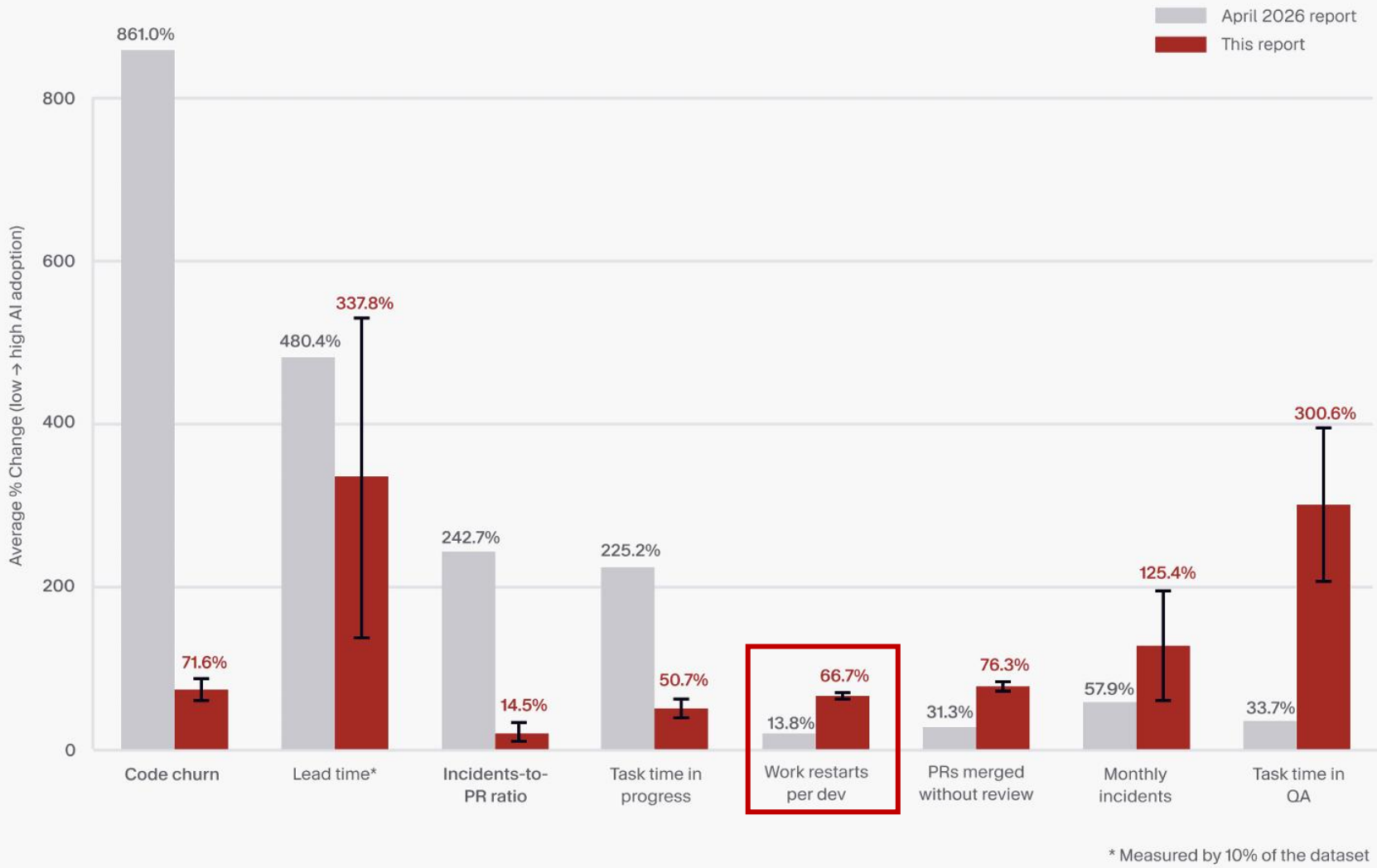
AI Engineering Report Q3 2026 | The Speed Trap

FARDS

ZURE

THE WHIPLASH, SIX MONTHS ON

Headline metrics: Prior report vs this report (% changes from low to high AI adoption)



Telemetry dataset size: 22,000 developers across 4,000 teams

FAROS

ZURE

THE COUNTERMEASURE WITH THE MOST EVIDENCE SO FAR

Effects of increased agentic review

LOW AGENTIC REVIEW

Median time to first review

+114.5%

longer under high AI adoption



PRs merged without review

+76.3%

cases where review is skipped entirely

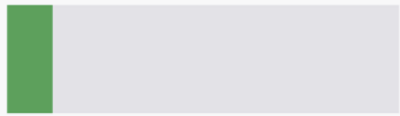


HIGH AGENTIC REVIEW

Median time to first review

-10.7%

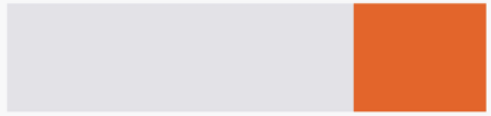
faster



PRs merged without review

+41.4%

still elevated, but less so



Telemetry dataset size: 22,000 developers across 4,000 teams

FAROS

ZURE

[Submitted on 25 Mar 2026 (v1), last revised 7 May 2026 (this version, v2)]

SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks

Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, Aws Albarghouthi

Software development is iterative, yet agentic coding benchmarks hide design issues through their single-shot setup. Recent iterative benchmarks attempt to remedy this but heavily constrain an agent's design decision space, making it impossible to faithfully measure how their decisions shape future extensions. We introduce SlopCodeBench, a benchmark of 36 problems and 196 checkpoints where agents repeatedly extend their own solutions. Unlike prior iterative benchmarks, our evolving specifications demand architectural decisions but leave internal structure to the agent. We measure two forms of degradation: structural erosion (concentrated complexity) and verbosity (redundant code). Evaluating 15 coding agents across open and closed models, we find that no agent fully solves any problem end-to-end, and the best agent passes 14.8% of checkpoints. Quality degrades across checkpoints, with structural erosion rising in 77% of trajectories and verbosity in 75.5%. Compared to 473 open-source Python repositories, agent code is 2.3x more verbose and 2.0x more eroded, and the human repositories degrade less often and by smaller margins across their git histories. Explicit quality guidance reduces initial verbosity and erosion by up to a third, without affecting degradation rates. SlopCodeBench provides the first measurement of code degradation under iterative extension, revealing that agents pass checkpoints while producing code that erodes and bloats with each turn.

Comments: Code and Leaderboards are located at this https URL
Subjects: **Software Engineering (cs.SE)**; Artificial Intelligence (cs.AI); Computation and Language (cs.CL)
Cite as: arXiv:2603.24755 **[cs.SE]**
(or arXiv:2603.24755v2 **[cs.SE]** for this version)
https://doi.org/10.48550/arXiv.2603.24755
Related DOI: https://doi.org/10.5281/zenodo.18405900,https://doi.org/10.5281/zenodo.19257129

[2603.24755] SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks

ZURE

Iterative Evaluation Is The Future

Aider and SWE-Bench evaluate an agent's ability to solve an issue given a frozen repository. Undoubtedly, this is an important capability, but this is a single point in time. An agent could produce an entirely viable, but utterly different from the ground truth, fix that would fundamentally change how a developer would solve the inevitable extension. Thus, measuring qualitative metrics at a single snapshot in time yields a noisy signal that is scaffolded by prior human decisions. Furthermore, agents are not evaluated on their performance in long-horizon coding tasks, where they must either live with or redesign their original choices. **Viewing agentic benchmarks as iterative processes is the only way to evaluate the true nature of software engineering.**

We must adopt this framing both now and for the future of agentic coding. Much of the recent discourse on agentic coding tools has focused on the "slop" they generate (i.e., **verbose comments**, **defensive coding**, **bloat**). While "slop" is ill-defined, the core of these grievances hits squarely on the

Model ▲	Harness ▲	Version ▲	Strict Solve ▼	Iso Solve ▲	$/CKPT ▲	Erosion ▲	Verbosity ▲
GPT 5.5 (High)	Codex	0.124.0	**14.29**	**28.06**	$1.51	0.494	0.269
GPT 5.3-Codex (High)	Codex	0.98.0	**11.22**	**26.02**	$0.69	0.644	0.336
GPT 5.4 (High)	Codex	0.110.0	**10.71**				
Kimi K2.6 (High)	Kimi CLI	1.37.0	**10.71**				
GPT 5.2-Codex (High)	Codex	0.93.0	**9.69**				
Opus 4.6 (High)	Claude Code	2.1.32	**9.69**				
GLM 5.1 (High)	Claude Code	2.1.44	**9.69**				
Opus 4.5 (High)	Claude Code	2.0.51	**9.18**	**17.35**	$2.53	0.691	0.297

14%

(the best tested model)

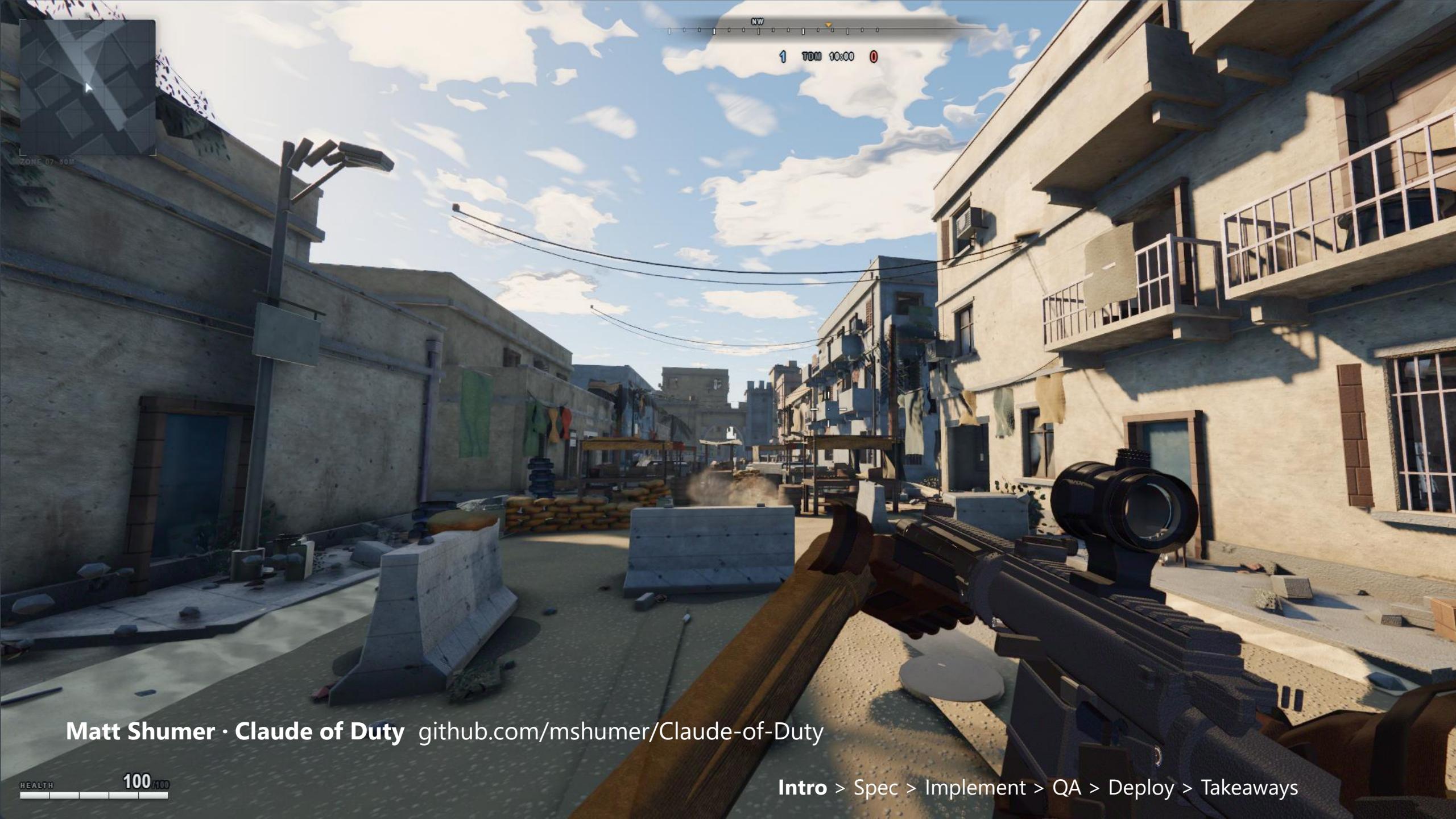
ZURE

Vibe coding:
Like peeing your pants in the winter

185k€

Zure CTO Sami's token spend, January to August 2026

~23k€ a month (fully offset by subscriptions and other funding, you think we have this kind of money?)



Matt Shumer · Claude of Duty github.com/mshumer/Claude-of-Duty

I want you to build a first-person shooter at the level of the most recent Call of Duty games. It should be utterly perfect, visually beautiful, with every single thing done at AAA quality—from textures to physics to anything you could think of.

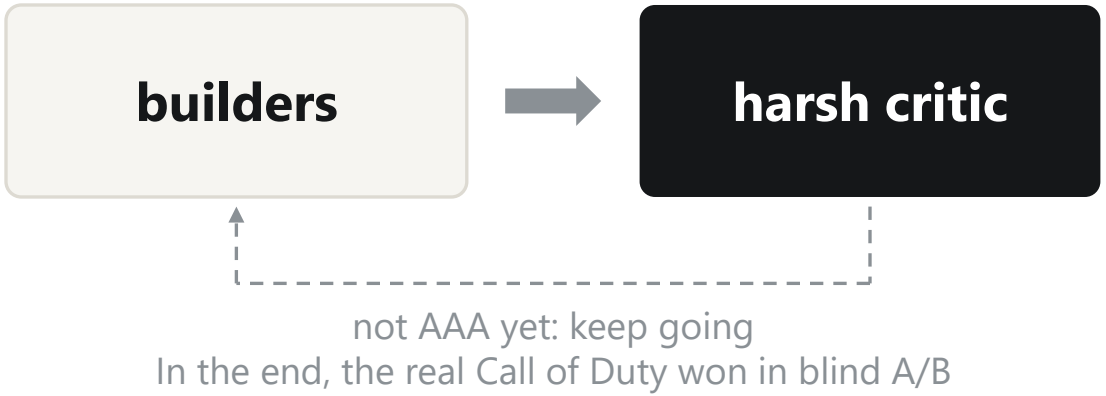
Fan out sub-agents and have sub-agents tackle each one individually so that the game is utterly perfect. You should /loop on each item and have a separate sub-agent check it visually to ensure it looks triple A. That separate sub-agent should be a really harsh critic, and if it doesn't look triple A, it should keep going.

Don't stop until each sub-agent is utterly wowed with the quality when compared with the actual Call of Duty game. It should literally compare them side by side blind and say which one looks better. Do this in ThreeJS. /loop until it's utterly perfect. Fan out sub-agents and ultracode.

Matt Shumer · Claude of Duty github.com/mshumer/Claude-of-Duty

Gauntlet Loop: Loop until a harsh critic is wowed

```
Fan out sub-agents …
/loop on each item and have a separate
sub-agent check it visually …
That separate sub-agent should be
a really harsh critic …
Don't stop until each sub-agent is
utterly wowed … compare them
side by side blind …
```



Matt Shumer · Claude of Duty github.com/mshumer/Claude-of-Duty

Had to try Gauntlet
Loop myself

Settings Status Config **Usage** Stats

Session

Total cost: $240.26
Total duration (API): 4h 48m 8s
Total duration (wall): 11h 32m 52s
Total code changes: 11286 lines added, 764 lines removed
Usage by model:
claude-haiku-5-5: 14.9k input, 877 output, 0 cache read, 0 cache write, 1 web search ($0.0119)
claude-opus-5-5: 1.1m input, 1.5m output, 560.2m cache read, 18.7m cache write ($240.25)
Prompt cache (main): 161 requests · 99% of input tokens from cache · no misses · cold — idle 6h 28m 33s, next turn re-caches the compacted prompt

Current session

0% used

Current week (all models)

10% used

Resets Oct 22, 7:59am (Europe/Helsinki)

Current week (Fable)

0% used

Resets Oct 22, 8am (Europe/Helsinki)

↓

ZURE

THE PROCESS

AI dev process

1 Spec → 2 Implement → 3 QA → 4 Deploy

1 Spec	2 Implement	3 QA	4 Deploy
Say what to build and why	The agent builds in small steps	The agent tests	Does your app use models, or just you?
spec-driven development **context engineering**	**harness engineering** **context engineering**	**agentic TDD** **testing & critic agents**	**harness engineering** **cost and context control**

Spec-driven development

The spec is the source of truth the agent builds from and is checked against.

Or should it be code, from where we generate the specs?

Context engineering

Choosing what goes into the model's window, and keeping everything else out. Minimize context sent to agent.

Harness engineering

Building the guides (rules, skills) and sensors (tests, linters, hooks) around the model.

Agentic TDD

The agent writes a failing test first, then code until it passes. The tests say when it's done. 80%+ coverage is easy.

Harness and skills run through all four: anything repeating or cumbersome can be a skill.

01
Spec

Say what to build and why

ZURE

Requirements gathering

Customer meetings are transcribed



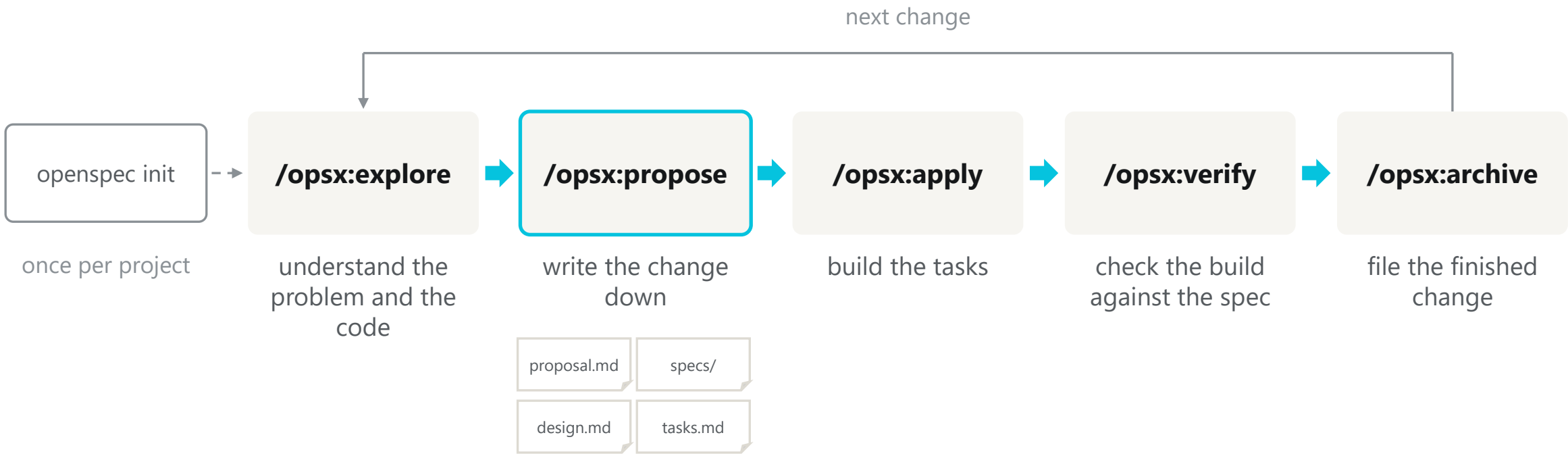
Skill captures content and writes them to project wiki and ADRs, filling answers for previously open questions.



Skill walks through backlog and either modifies or adds new items (with labels) as necessary

One of my lazy colleague's requirements gathering process.

OpenSpec is a light spec-driven dev framework



npm install -g @fission-ai/openspec@latest

Spending time in planning saves time in reviewing code

02
Implement

The agent builds in small steps

ZURE

slopalytics Benchmarks Usage | Claude vs Codex Models Harnesses

Claude Code vs Codex

usage data from T3 Code

Updated 8 Oct, 05.43

Prompts Tokens Est. spend | 7 days 14 days 30 days

Claude Code

67.0%

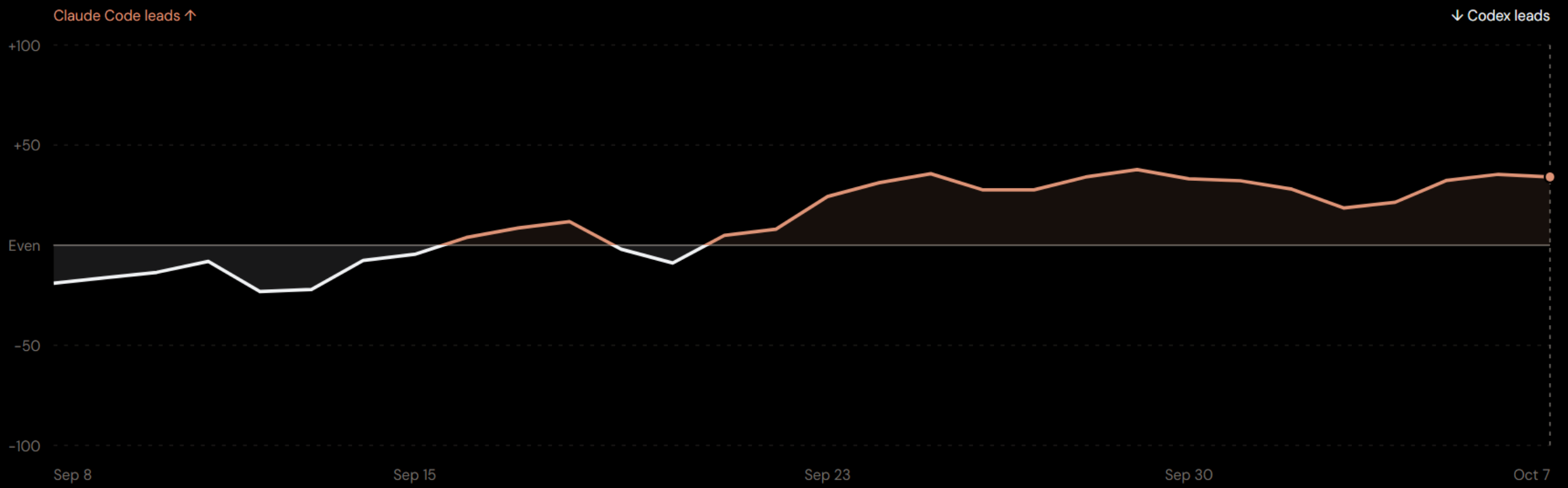
Oct 7

+34.1 pp

Claude leads

Codex

33.0%



0 = 50/50 · +20 = Claude 60%, Codex 40%

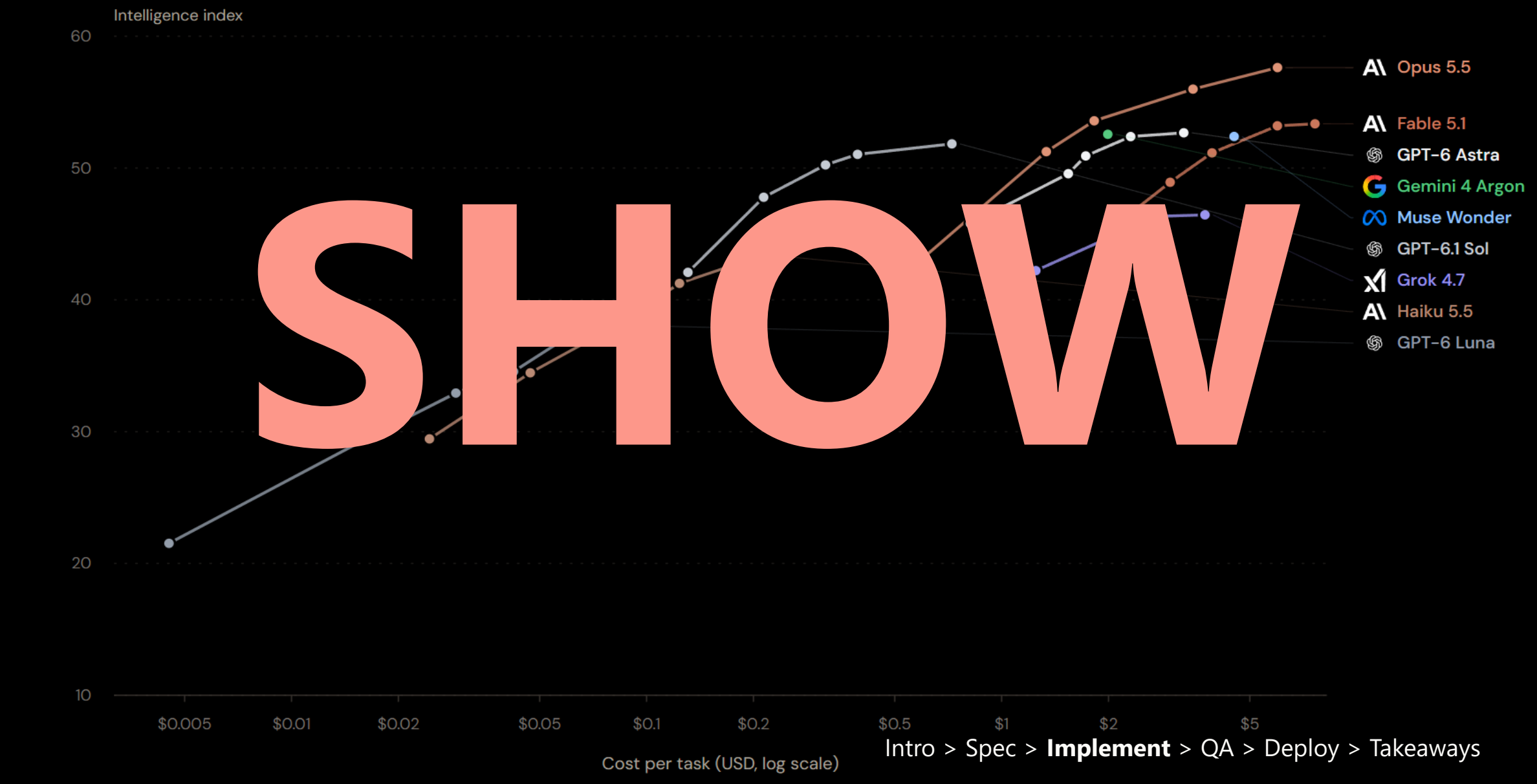
Intro > Spec > **Implement** > QA > Deploy > Takeaways

53.2pp toward Claude since Sep 8

slopalytics Benchmarks Usage | Cost vs intelligence | Time per task vs intelligence | Tokens per task | Total tokens | Cost per task | Total cost

Cost vs intelligence

Pareto line | Linear | Log

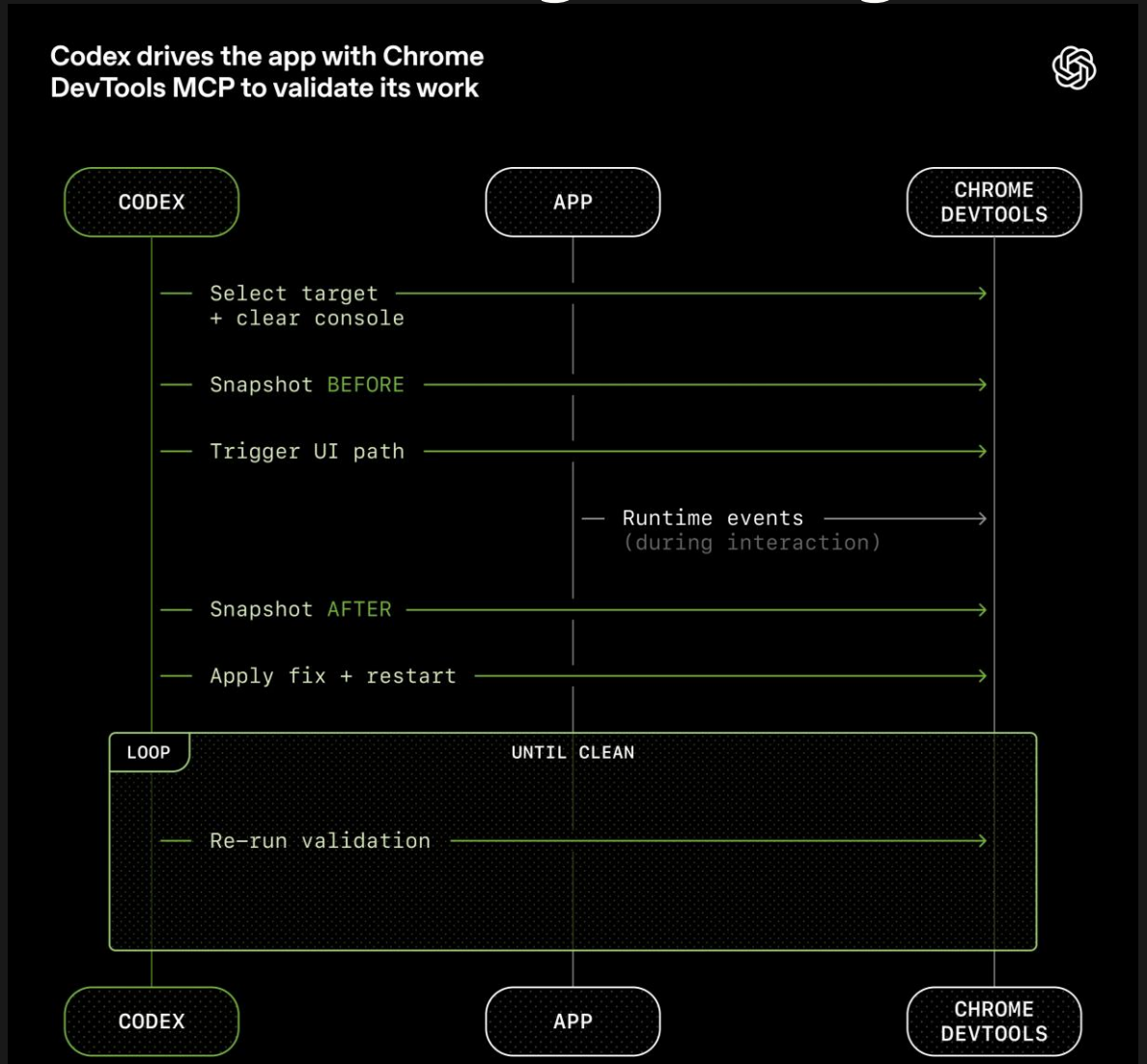


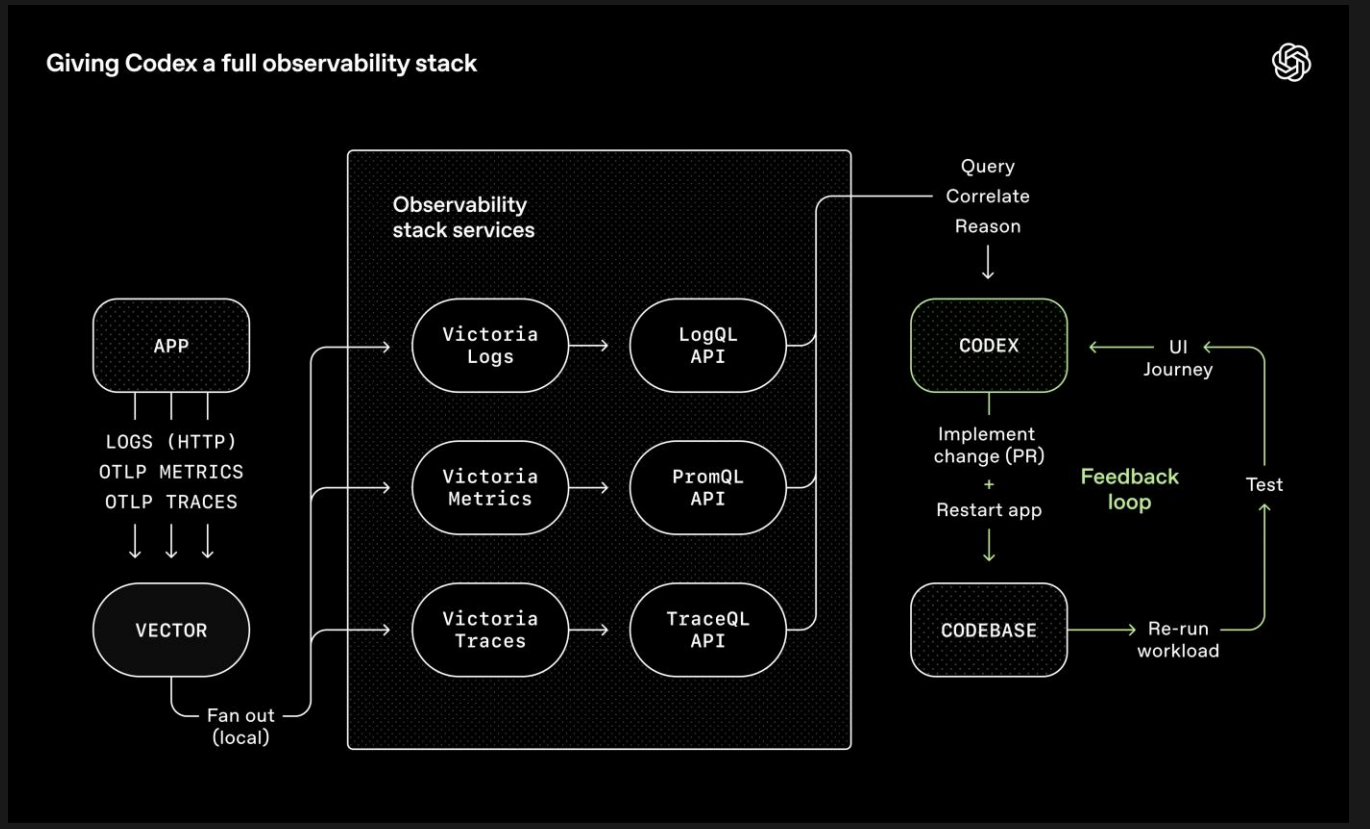
SHOW

Intro > Spec > **Implement** > QA > Deploy > Takeaways

02 · IMPLEMENT

Harness engineering





Ryan Lopopolo · Harness Engineering (11th Feb, 2026)
https://openai.com/index/harness-engineering/

The primary job of our engineering team became enabling the agents to do useful work.

In practice, this meant working depth-first: breaking down larger goals into smaller building blocks (design, code, review, test, etc), prompting the agent to construct those blocks, and using them to unlock more complex tasks.

ZURE

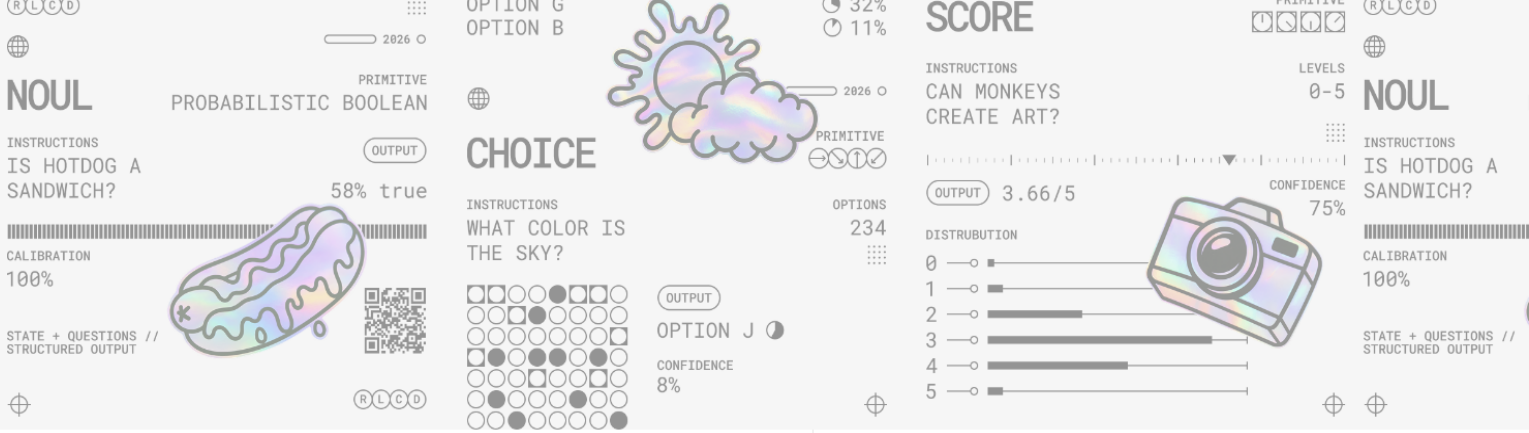
VIEW EXAMPLE REQUESTS

Learn to TypeSafe

Playground →

JEV V13
SYSTEM ONE

~~RLHF~~ RLCD



TypeSafe in action

Parallel questions

Runs a 13-question regulatory briefing over the GDPR Wikipedia article, showing that batching every question into one TypeSafe call is 11.5x cheaper and 9.6x faster with no change in answers.

SDE cascade

Uses a 2-stage structured-data-extraction cascade (mini → verify → reasoning) to get most of the quality of a big reasoning model at a fraction of the cost.

Self-consistency

Scores an auto-insurance claim against a 15-question adjudication rubric, showing that LLM answers vary run to run while TypeSafe returns stable noul probabilities far faster and cheaper.

Function calling

Turns natural-language trading requests into calls to ordinary typed functions by mapping function names and closed-set arguments to confidence-aware TypeSafe questions.

All cookbooks ↗



Wikirace
How many hops will TypeSafe take?



Smarthome
TypeSafe controls appliances

Quickstart

Agent setup ↗

```
Install the TypeSafe skill. If you're in
Claude Code, run `claude plugin marketplace
add typesafe-ai/skills`, then `claude plugin
install typesafe@typesafe-ai`. If you're in
another agent, run `npx ski
ai/skills --skill typesafe
```

Copy Agent Prompt

SKILL.md — View

API key

Documentation

Usage — View

Stats may be delayed

Tokens
1.8M



$0.042/MTok input · Free output

Balance
$24.92

SN Sakari Nahi
Sakari Org

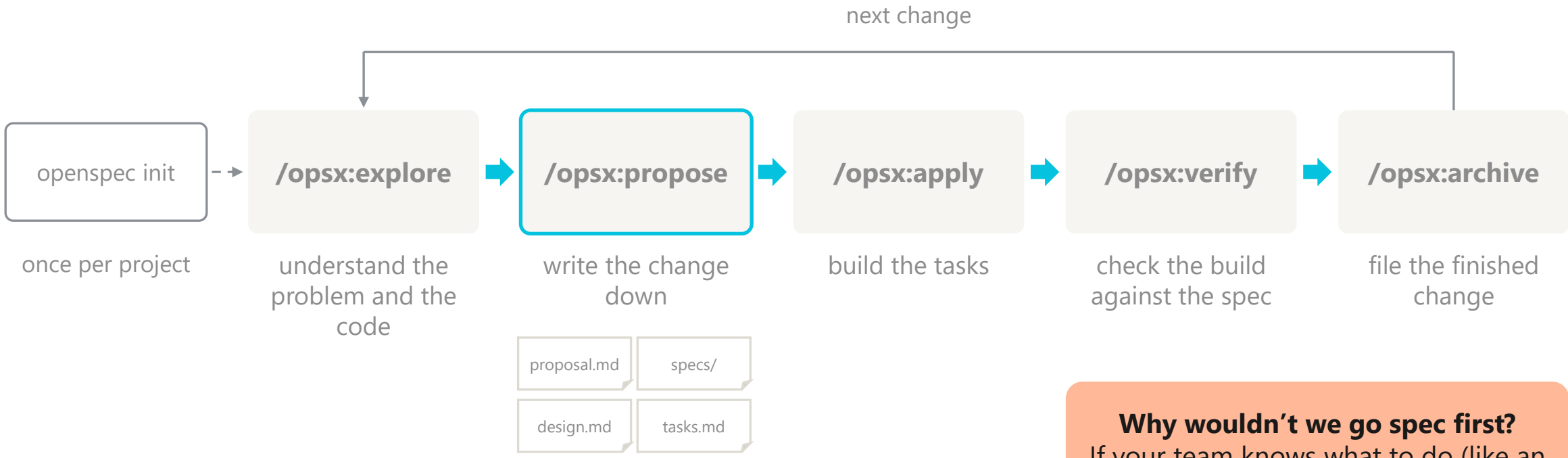
Our Data Policy
Trust Center
Replay intro
Join us on Discord

Intro > Spec > **Implement** > QA > Deploy > Takeaways

Help us improve Jev

OpenSpec usage

If I want to go spec-first, why would I write this many commands?



Why wouldn't we go spec first?
If your team knows what to do (like an elite product-specific team), have everyone generate code and pull the documentation from code when asked.

npm install -g @fission-ai/openspec@latest

Five agents with the same idea, prompt and model

BASELINE

raw-low	raw-med	ecc-med	firstmate-med	matt-med
no skills · low	no skills · medium	ECC · medium	firstmate · medium	Pocock · medium
:3001	:3002	:3003	:3004	:3005

effort

skill pack

-> Scoreboard.sh

— V2.0 · THE SYSTEM, NOT THE CONFIGS

The operating system for AI agent harnesses.

Skills, instincts, memory, security scanning, and research-first development — a complete system, **not a config pack**.

RUNS ACROSS

Claude Code | Codex | Cursor | OpenCode | Gemini | Zed | Copilot

Affaan Mustafa · Everything Claude Code
https://github.com/affaan-m/ecc

SKILLS 261	AGENTS 64	COMMANDS 84
agent-harness-constr…	harness-optimizer	/harness-audit
context-budget	code-reviewer	/code-review
continuous-agent-loop	code-architect	/feature-dev
agentic-os	code-simplifier	/checkpoint
autonomous-loops	gan-planner	/gan-design
mcp-server-patterns	gan-generator	/evolve
ai-regression-testing	gan-evaluator	/hookify
agent-eval	e2e-runner	/santa-loop
content-hash-cache-p…	build-error-resolver	/security-scan
bun-runtime	a11y-architect	/skill-create
codebase-onboarding	database-reviewer	/model-route
nextjs-turbopack	doc-updater	/quality-gate
+ 249 more	+ 52 more	+ 72 more

409 CATALOG **7** HARNESSES **ECC 2.0α** RUST PLANE **MIT**

NPM: **ECC-UNIVERSAL** · AGENTSHIELD

Firstmate

Talk to one agent. Orchestrate many crewmates across your repos.



tmux sessions

- Crewmate A — worktree 1 — repo: web-app
- Crewmate B — worktree 2 — repo: web-app
- Crewmate C — worktree 3 — repo: backend
- Crewmate D — worktree 4 — repo: mobile

same repo: web-app

repo: backend

repo: mobile

Each crewmate runs in its own worktree. Multiple worktrees can come from the same repo.

REAL AI ENGINEERING · BY MATT POCOCK

Engineering fundamentals aren't obsolete.

They're your biggest advantage. AI Hero is the engineering process for working with coding agents, from an idea to shipped, reviewed code.

Start the free email course | Browse the skills →

131,400+ DEVELOPERS LEARNING

8,500+ TRAINED IN COHORTS

27 FREE SKILLS



Matt Pocock · Matt Pocock's Skills
https://www.aihero.dev/

npx skills@latest add mattpocock/skills --skill=setup-matt-pocock-skills

A battle royale in the browser. Everyone who opens the page, on a phone or a laptop, drops into the same arena with a generated name: **a 30 by 30 grid seen from above**.

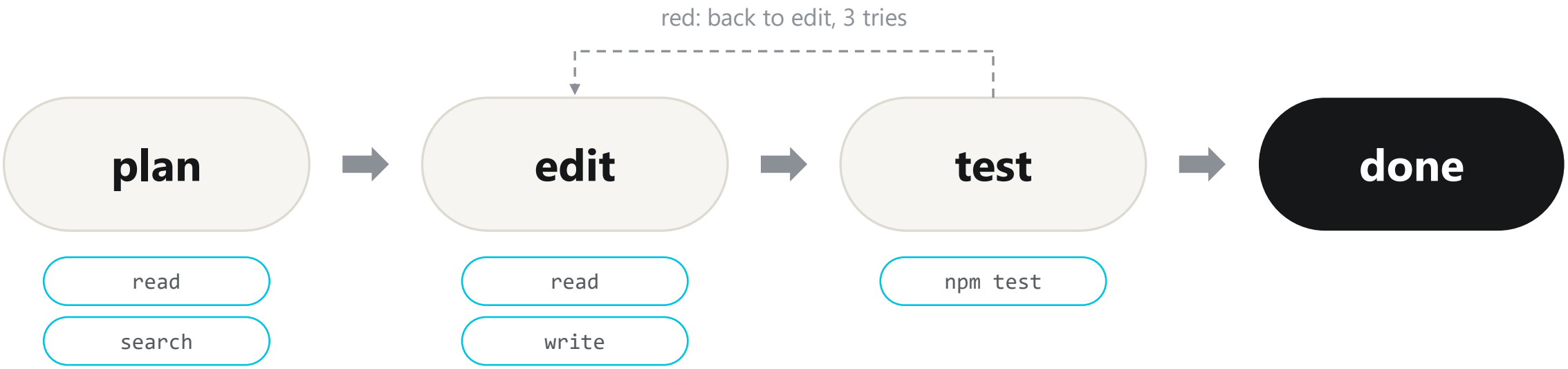
You run around and hit the other players. The safe zone shrinks over time, and outside it you get hurt. The last one standing wins, and then a new round starts.

If you're the only one there, a computer player shows up so you have someone to fight. Kills, staying alive and winning all earn points, and there's a live highscore list. You can see everyone's health, the damage you deal and take, when you can strike again, and the rules.

It should look sharp and feel alive, in exciting, dynamic colours: smooth movement, hits you can feel, a zone that visibly closes in. If you die you can still watch. If you join in the middle of a round, you wait for the next one.

ZURE

State-driven process



Examples: Statewright & Stately
Enforced per state, not asked for in the prompt

local model: fail → 10/10

13.8 GB model, 5 SWE-bench tasks, vendor's figures

03
QA

Through harness, agent tests

ZURE

8%

In 2020, wrote tests first

Diffblue developer survey



“Use red/green TDD.”

Simon Willison

ZURE

Is our OpenSpec testing itself?

Do we have scores from our skill pack tests already?

04
Deploy

Yay \o/ we're in production, no more work

Minimizing Context

Thoughts:

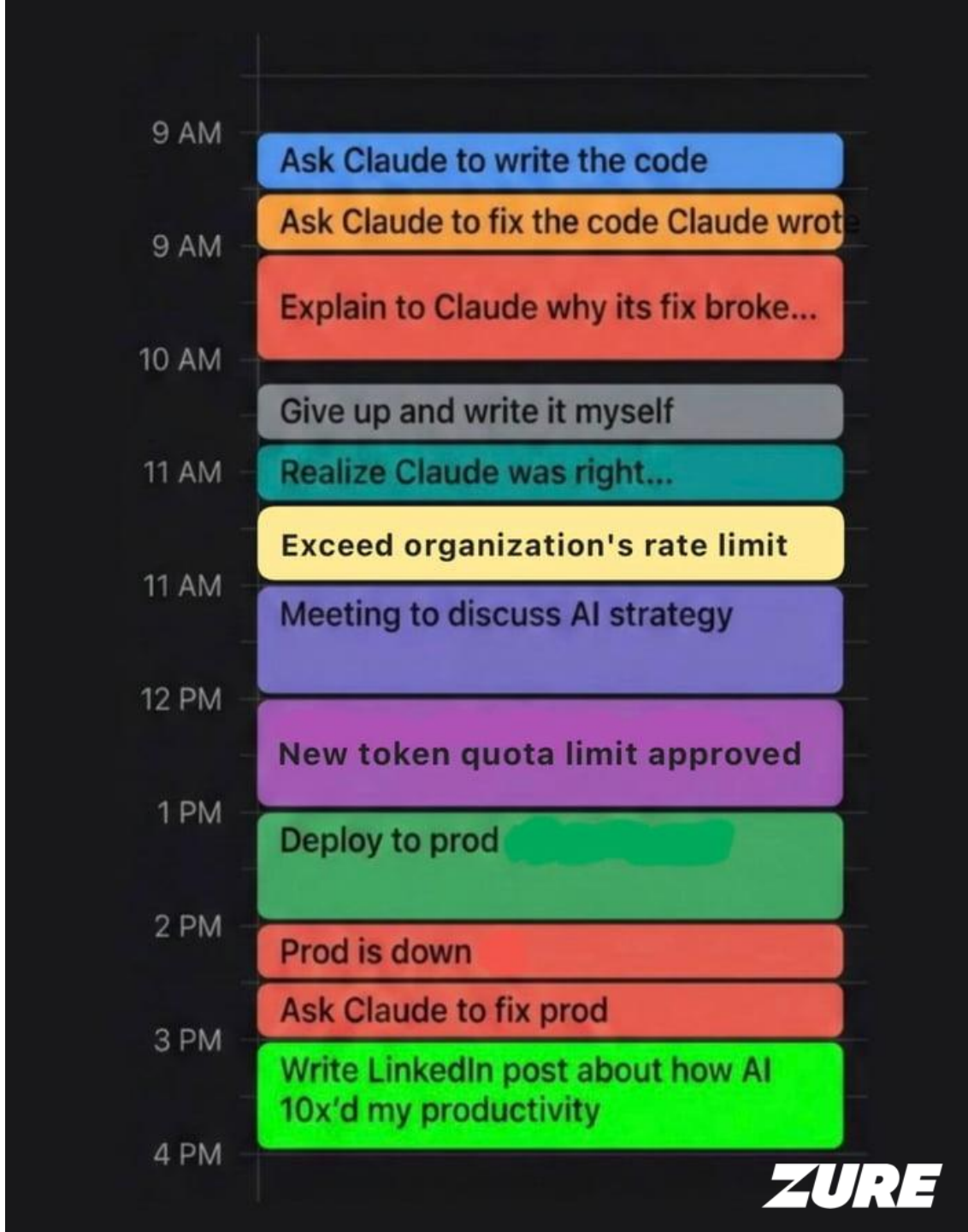
- Context-mode (demo)
- Source MCP layer (Sourcebot/Sourcegraph)
- Minimize context for your agent
  - For example, for each test, have its execution log be in a separate file, so agents debugging only need to read the one file

Apps in production: simple dev task future

(CraftAgent demo)

Takeaways

- Harness engineering is part of the answer; making something non-deterministic more deterministic
- Whatever the source, understanding requirements is becoming even more important than before
- To enable quality, our job will be to
    a) Write the spec of features and architecture to the right degree
    b) Build the app harness to enable AI to
    c) Access everything
    d) While minimizing context (and costs)
    e) And code reviewing formative and critical parts
- In the future QA is key, devote time to figure out the sensors for the quality of your solution
- With AI, the digitalization will speed up, and technologists will be more in demand than ever





Plz
feedback

ZURE